PRÁCTICA SESIÓN 19
Métricas de Calidad: Complejidad Ciclomática y CRAP Score

Cómo ver la Complejidad Ciclomática (CC)
IntelliJ por defecto NO la muestra directamente.

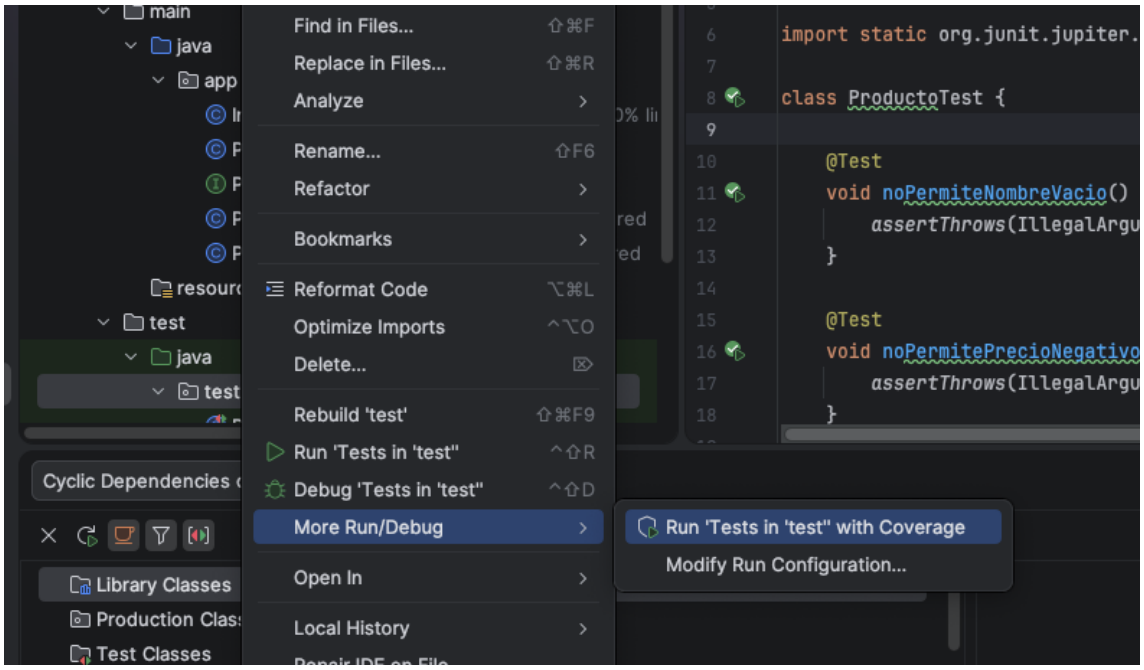
Instalar SonarQube

Paso 1
Instalar.

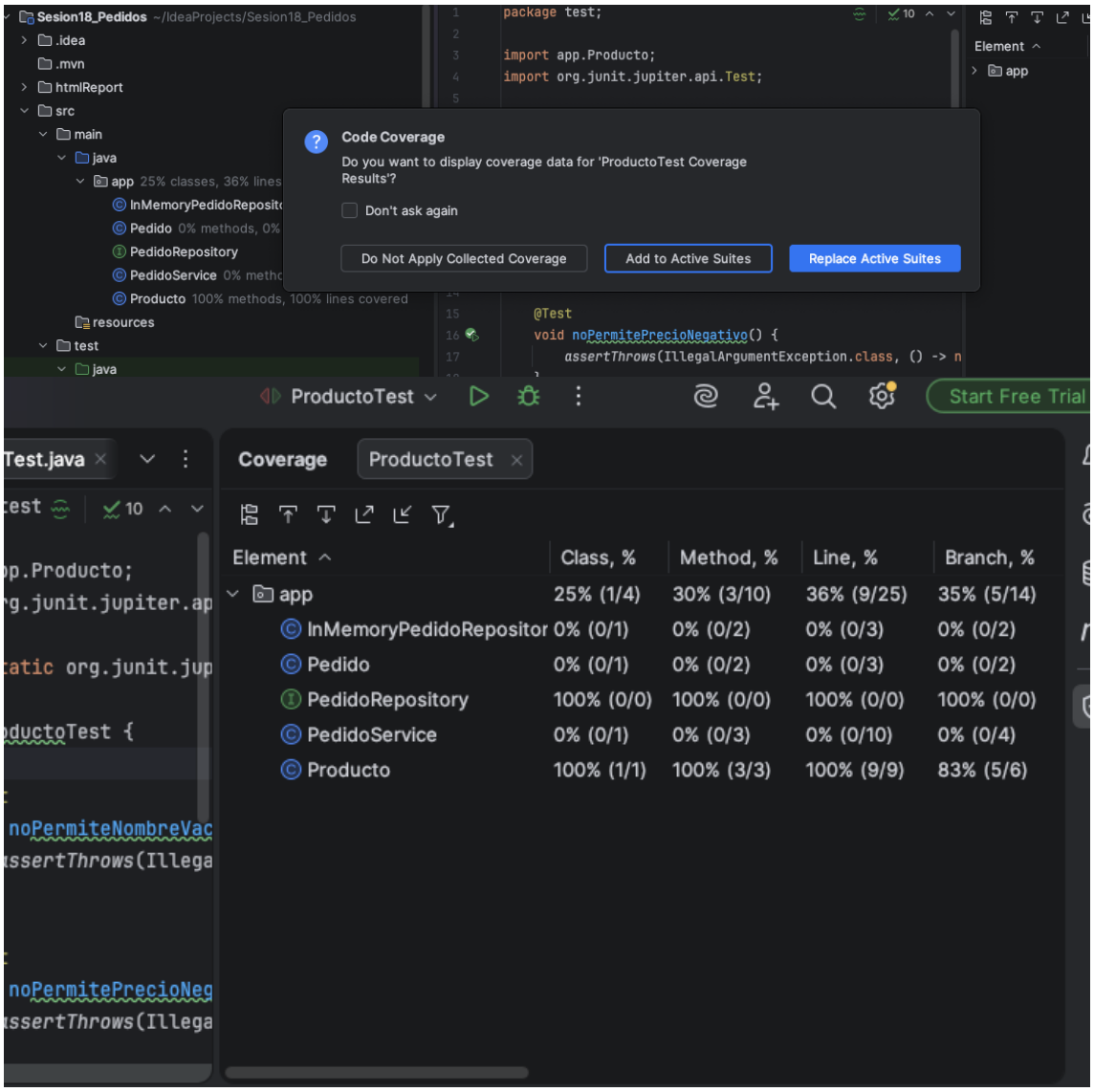
Paso 2
Busca:
SonarQube
Instálalo y reinicia.

Paso 3

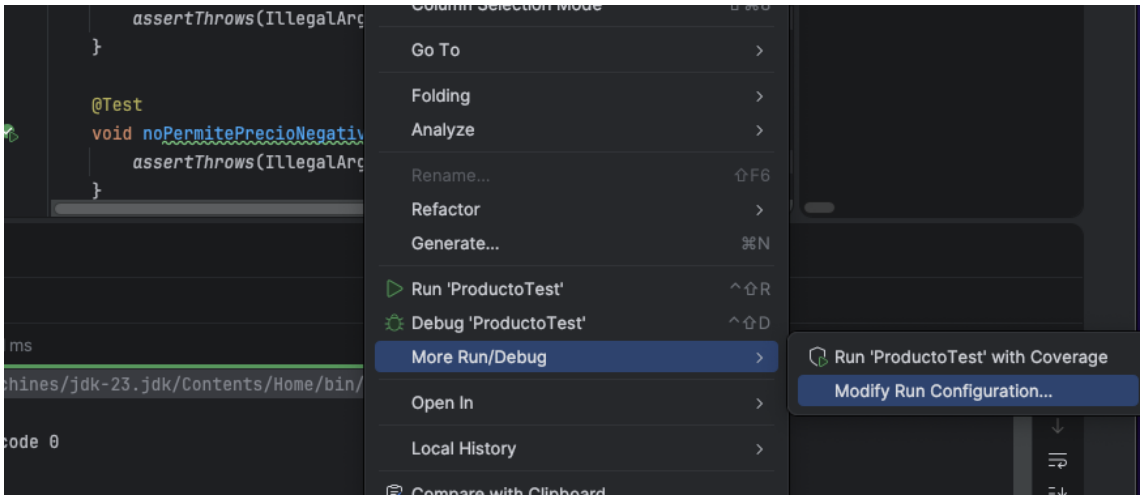
Botón derecho Analyze → Analyze with SonarQube/Coverage



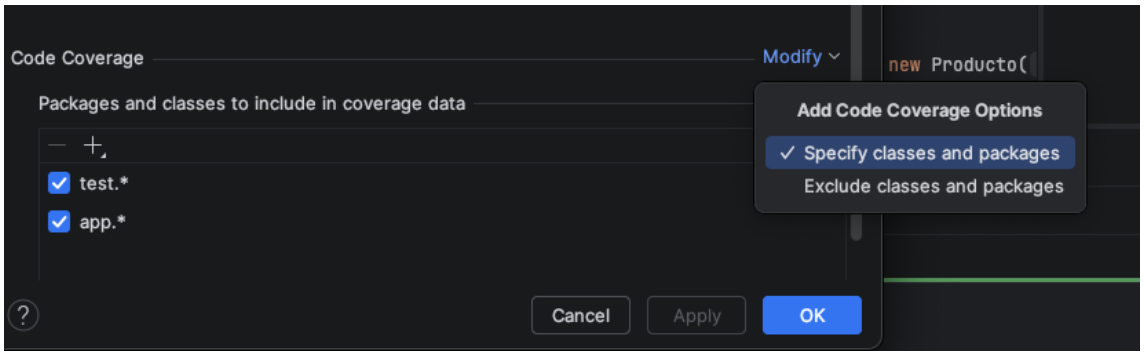
Damos a Replace



Si no os sale nada,



En Modify añadir vuestro paquete, el mío es app.



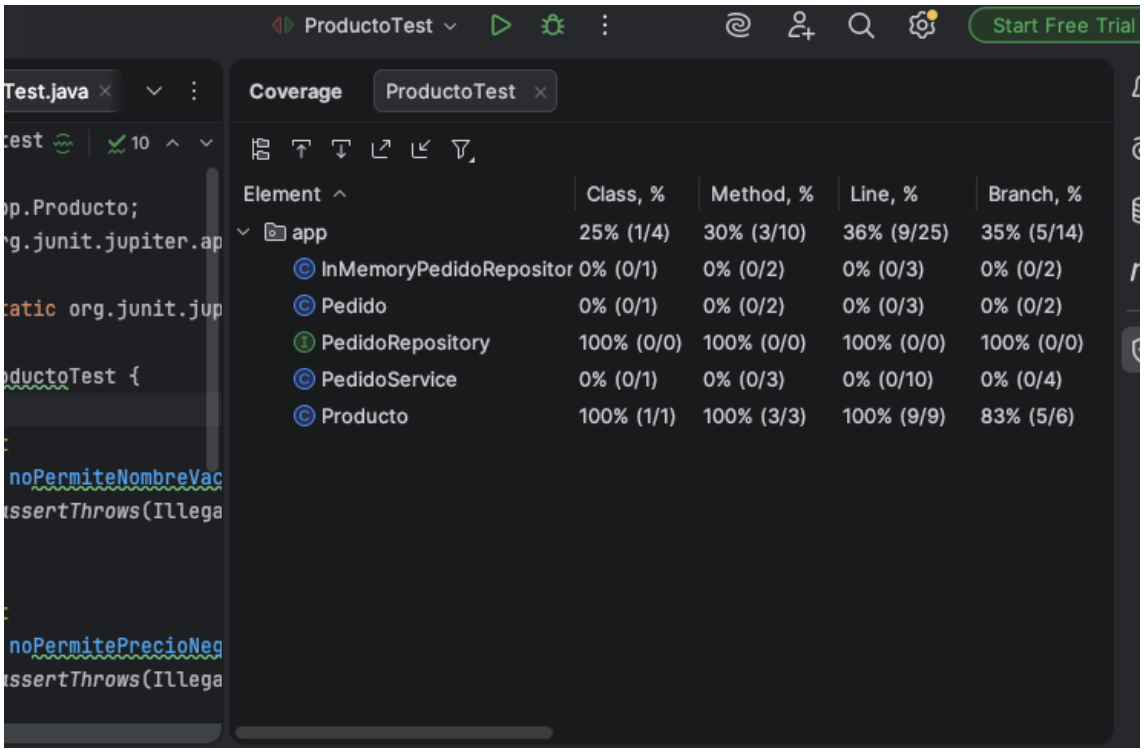
Os dejo mi pom.xml

Sin JaCoCo puede que no funcione.
JaCoCo (Java Code Coverage) es una herramienta que sirve para medir la cobertura de código en proyectos Java.
Te dice qué porcentaje de tu código está siendo ejecutado por tus tests.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>

   <groupId>org.example</groupId>
   <artifactId>Sesion18_Pedidos</artifactId>
   <version>1.0-SNAPSHOT</version>

   <properties>
      <maven.compiler.source>23</maven.compiler.source>
      <maven.compiler.target>23</maven.compiler.target>
      <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
   </properties>

   <dependencies>
      <dependency>
```

```
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.10.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.2.5</version>
        </plugin>
            <plugin>
                <groupId>org.jacoco</groupId>
                <artifactId>jacoco-maven-plugin</artifactId>
                <version>0.8.8</version>
                <executions>
                    <execution>
                        <goals>
                            <goal>prepare-agent</goal>
                        </goals>
                    </execution>
                    <execution>
                        <id>report</id>
                        <phase>test</phase>
                        <goals>
                            <goal>report</goal>
                        </goals>
                    </execution>
                </executions>
            </plugin>


    </plugins>
</build>
```

</project>



¿Qué significa cada columna?

Class %

Indica qué porcentaje de clases han sido ejecutadas por los tests.

Producto: 100% (1/1)
Pedido: 0% (0/1)

Significa:

- Producto: la clase se ejecutó
- Pedido: nunca fue usada en ningún test

El formato (1/1) significa:

- 1 clase cubierta
- 1 clase total

Method %

Indica qué porcentaje de métodos se han ejecutado al menos una vez.

Ejemplo:
Producto: 100% (3/3)
Significa:

- Tiene 3 métodos
- Los 3 fueron ejecutados por los tests

Si pone:
0% (0/2)

Significa:

- Tiene 2 métodos
- Ninguno fue llamado por ningún test

Un método cuenta como cubierto si se ejecuta al menos una vez.

Line %

Indica qué porcentaje de líneas de código fueron ejecutadas.

Ejemplo:

Producto: 100% (9/9)

Significa:

- 9 líneas ejecutables
- Las 9 se ejecutaron

Si ves:
83% (5/6)
Significa:

- 6 líneas ejecutables
- 5 fueron ejecutadas
- 1 nunca se ejecutó

Esa línea suele estar marcada en rojo en el editor.

Branch %

Branch = caminos lógicos.

Cada if, else, switch, while genera ramas.

Ejemplo:

```
if (precio < 0) {
   throw new IllegalArgumentException();
}
```

Ese if tiene 2 ramas:

- Rama TRUE
- Rama FALSE

Si tu test solo prueba un caso:
producto.setPrecio(10);
Solo se ejecuta la rama FALSE.

Entonces:
Branch: 50% (1/2)

Si pruebas también:
producto.setPrecio(-5);
Entonces:
Branch: 100% (2/2)

En la captura tenemos:

Producto: Branch 83% (5/6)
Significa:

- Hay 6 caminos posibles
- Solo 5 han sido ejecutados
- Te falta probar una condición

Seguramente:

- No has probado una excepción

- O no has probado un caso límite

¿Por qué Branch es más importante que Line?

Porque puedes tener:
- Line: 100%
- Branch: 50%

Eso significa:
- Todas las líneas se ejecutaron
- Pero no todos los caminos lógicos

Y ahí es donde aparecen los bugs en producción.

Resumen

En empresa, lo que más miran es:
- Cobertura de líneas
- Cobertura de ramas (branch coverage)

Interpretación global del proyecto según la captura:
- Producto: bien cubierto
- Pedido, PedidoRepository, etc: no tienen tests
- El repositorio InMemoryPedidoRepository está al 0%

Eso significa que solo estás testeando Producto.

¿Qué deberías hacer ahora?

Para subir el Branch de Producto al 100%:
1. Revisa los if que tenga
2. Escribe tests para:
    - Casos válidos
    - Casos inválidos
    - Valores límite
    - Excepciones